torch.sparse_coo_tensor#

https://pytorch.org/docs/stable/generated/torch.sparse_coo_tensor.html#torch

Description:

Constructs a sparse tensor in COO(rdinate) format with specified values at the given indices. This function returns an uncoalesced tensor when is_coalesced is unspecified or None. If the device argument is not specified the device of the given values and indices tensor(s) must match. If, however, the argument is specified the input Tensors will be converted to the given device and in turn determine the device of the constructed sparse tensor.

Function Signature

```
torch.sparse_coo_tensor(indices, values, size=None, *,
dtype=None, device=None, pin_memory=False,
requires_grad=False, check_invariants=None,
is_coalesced=None) → Tensor#

Parameters

Keyword Arguments
```

Parameters

Keyword

dtype (torch.dtype, optional) – the desired data type of returned tensor. Default: if None, infers data type from values. device (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see torch.set_default_device()). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types. pin_memory (bool, optional) – If set, returned tensor would be allocated in the pinned memory. Works only for CPU tensors. Default: False. requires_grad (bool, optional) – If autograd should record operations on the returned tensor. Default: False. check_invariants (bool, optional) – If sparse tensor invariants are checked. Default: as returned by torch.sparse.check_sparse_tensor_invariants.is_enabled()

Code Examples

```
>>> i = torch.tensor([[0, 1, 1],
...                   [2, 0, 2]])
>>> v = torch.tensor([3, 4, 5], dtype=torch.float32)
>>> torch.sparse_coo_tensor(i, v, [2, 4])
tensor(indices=tensor([[0, 1, 1],
                       [2, 0, 2]]),
       values=tensor([3., 4., 5.]),
       size=(2, 4), nnz=3, layout=torch.sparse_coo)

>>> torch.sparse_coo_tensor(i, v)  # Shape inference
```

```
tensor(indices=tensor([[0, 1, 1],
                       [2, 0, 2]]),
       values=tensor([3., 4., 5.]),
       size=(2, 3), nnz=3, layout=torch.sparse_coo)

>>> torch.sparse_coo_tensor(i, v, [2, 4],
...                         dtype=torch.float64,
...                         device=torch.device('cuda:0'))
tensor(indices=tensor([[0, 1, 1],
                       [2, 0, 2]]),
       values=tensor([3., 4., 5.]),
       device='cuda:0', size=(2, 4), nnz=3, dtype=torch.float64,
       layout=torch.sparse_coo)

# Create an empty sparse tensor with the following invariants:
#   1. sparse_dim + dense_dim = len(SparseTensor.shape)
#   2. SparseTensor._indices().shape = (sparse_dim, nnz)
#   3. SparseTensor._values().shape = (nnz,
SparseTensor.shape[sparse_dim:])
#
# For instance, to create an empty sparse tensor with nnz = 0,
dense_dim = 0 and
# sparse_dim = 1 (hence indices is a 2D tensor of shape = (1,
0))
>>> S = torch.sparse_coo_tensor(torch.empty([1, 0]), [], [1])
tensor(indices=tensor([], size=(1, 0)),
       values=tensor([], size=(0,)),
       size=(1,), nnz=0, layout=torch.sparse_coo)

# and to create an empty sparse tensor with nnz = 0, dense_dim =
1 and
# sparse_dim = 1
>>> S = torch.sparse_coo_tensor(torch.empty([1, 0]),
torch.empty([0, 2]), [1, 2])
tensor(indices=tensor([], size=(1, 0)),
       values=tensor([], size=(0, 2)),
       size=(1, 2), nnz=0, layout=torch.sparse_coo)
```

Notes & Warnings

NOTE Note This function returns an uncoalesced tensor when is_coalesced is unspecified or None.

NOTE Note If the device argument is not specified the device of the given values and indices tensor(s) must match. If, however, the argument is specified the input Tensors will be converted to the given device and in turn determine the device of the constructed sparse tensor.

Detailed Documentation

Documentation

Constructs a sparse tensor in COO(rdinate) format with specified values at the given indices.

This function returns an uncoalesced tensor when is_coalesced is unspecified or None.

If the device argument is not specified the device of the given values and indices tensor(s) must match. If, however, the argument is specified the input Tensors will be converted to the given device and in turn determine the device of the constructed sparse tensor.

indices (array_like) – Initial data for the tensor. Can be a list, tuple, NumPy ndarray,

scalar, and other types. Will be cast to a torch.LongTensor internally. The indices are the coordinates of the non-zero values in the matrix, and thus should be two-dimensional where the first dimension is the number of tensor dimensions and the second dimension is the number of non-zero values.

values (array_like) – Initial values for the tensor. Can be a list, tuple, NumPy ndarray, scalar, and other types.

size (list, tuple, or torch.Size, optional) – Size of the sparse tensor. If not provided the size will be inferred as the minimum size big enough to hold all non-zero elements.

dtype (torch.dtype, optional) – the desired data type of returned tensor. Default: if None, infers data type from values.

device (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see torch.set_default_device()). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types.

pin_memory (bool, optional) – If set, returned tensor would be allocated in the pinned memory. Works only for CPU tensors. Default: False.

requires_grad (bool, optional) – If autograd should record operations on the returned tensor. Default: False.

check_invariants (bool, optional) – If sparse tensor invariants are checked. Default: as returned by torch.sparse.check_sparse_tensor_invariants.is_enabled(), initially False.

is_coalesced (bool, optional) – When``True``, the caller is responsible for providing tensor indices that correspond to a coalesced tensor. If the check_invariants flag is False, no error will be raised if the prerequisites are not met and this will lead to silently incorrect results. To force coalescion please use coalesce() on the resulting Tensor.

Default: None: except for trivial cases (e.g. nnz < 2) the resulting Tensor has is_coalesced set to False`.
